

Task 3: Robotic Arm Kinematics Assembly & Path Trace

Robotics & Automation Internship — Progree

1. Objective & System Specifications

This report documents the structural kinematic assembly, forward transformation mapping, and inverse joint solver for a 3-Degree-of-Freedom (3-DOF) planar robotic manipulator intended for industrial pick-and-place sorting operations.

- Degrees of Freedom (DOF): 3 Revolute Joints (θ_1 , θ_2 , θ_3)
- Link Dimensions: Link 1 (L_1) = 10.0 cm, Link 2 (L_2) = 8.0 cm, Link 3 (L_3) = 5.0 cm
- Total Max Reach: $L_1 + L_2 + L_3 = 23.0$ cm

2. Forward Kinematics (FK) Mathematical Derivation

Forward Kinematics maps given joint angles $[\theta_1, \theta_2, \theta_3]$ to the Cartesian spatial position (X , Y) and end-effector tool orientation (Φ):

$$\begin{aligned} X &= L_1 * \cos(\theta_1) + L_2 * \cos(\theta_1 + \theta_2) + L_3 * \cos(\theta_1 + \theta_2 + \theta_3) \\ Y &= L_1 * \sin(\theta_1) + L_2 * \sin(\theta_1 + \theta_2) + L_3 * \sin(\theta_1 + \theta_2 + \theta_3) \\ \Phi &= \theta_1 + \theta_2 + \theta_3 \end{aligned}$$

Example: For angles $\theta_1=35^\circ$, $\theta_2=45^\circ$, $\theta_3=-30^\circ$:

- $X = 10*\cos(35^\circ) + 8*\cos(80^\circ) + 5*\cos(50^\circ) = 8.19 + 1.39 + 3.21 = 12.79$ cm
- $Y = 10*\sin(35^\circ) + 8*\sin(80^\circ) + 5*\sin(50^\circ) = 5.74 + 7.88 + 3.83 = 17.45$ cm
- Tool Orientation: $\Phi = 35^\circ + 45^\circ - 30^\circ = 50.0^\circ$

3. Inverse Kinematics (IK) Algebraic Solution

To perform pick-and-place, the controller solves for joint angles given a target coordinate (X_{target} , Y_{target}) and desired tool angle Φ :

```
1. Wrist Center Coordinates:
  Wx = X_target - L3 * cos(Φ)
  Wy = Y_target - L3 * sin(Φ)

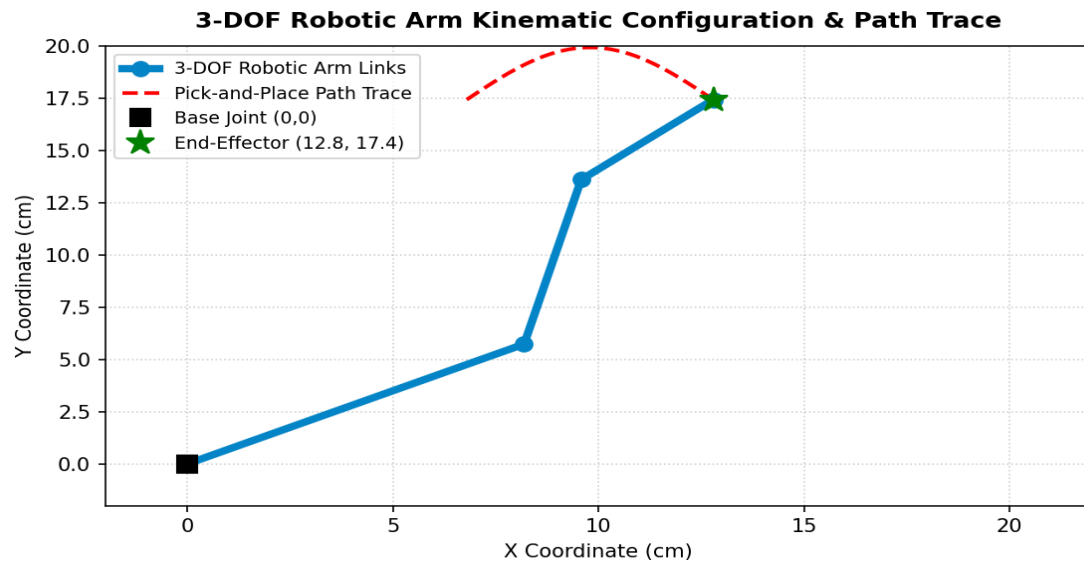
2. Law of Cosines for Joint 2:
  cos(θ2) = (Wx² + Wy² - L1² - L2²) / (2 * L1 * L2)
  θ2 = atan2(√(1 - cos²(θ2)), cos(θ2))

3. Joint 1 Angle:
  θ1 = atan2(Wy, Wx) - atan2(L2 * sin(θ2), L1 + L2 * cos(θ2))

4. Joint 3 Angle:
  θ3 = Φ - (θ1 + θ2)
```

4. Kinematic Assembly & Trajectory Plot

The kinematic chain and continuous pick-and-place arc trajectory are visualized below:



5. Python Kinematic Engine Script

Complete standalone script implemented for kinematics verification:

```

# Progree Robotics & Automation – Task 3
# Beginner 3-DOF Robotic Arm Kinematics Calculator (Forward & Inverse)

import math

class RoboticArm3DOF:
    def __init__(self, L1=10.0, L2=8.0, L3=5.0):
        self.L1 = L1
        self.L2 = L2
        self.L3 = L3

    def forward_kinematics(self, theta1_deg, theta2_deg, theta3_deg):
        t1 = math.radians(theta1_deg)
        t2 = math.radians(theta2_deg)
        t3 = math.radians(theta3_deg)

        x = self.L1 * math.cos(t1) + self.L2 * math.cos(t1 + t2) + self.L3 * math.cos(t1 + t2 + t3)
        y = self.L1 * math.sin(t1) + self.L2 * math.sin(t1 + t2) + self.L3 * math.sin(t1 + t2 + t3)
        phi = theta1_deg + theta2_deg + theta3_deg
        return round(x, 2), round(y, 2), round(phi, 2)

    def inverse_kinematics(self, target_x, target_y, phi_deg=0.0):
        phi = math.radians(phi_deg)
        wx = target_x - self.L3 * math.cos(phi)
        wy = target_y - self.L3 * math.sin(phi)

        D = (wx**2 + wy**2 - self.L1**2 - self.L2**2) / (2 * self.L1 * self.L2)
        if abs(D) > 1.0:
            return None

        theta2_rad = math.atan2(math.sqrt(1 - D**2), D)
        theta1_rad = math.atan2(wy, wx) - math.atan2(self.L2 * math.sin(theta2_rad), self.L1 + self.L2 * math.cos(theta2_rad))
        theta3_rad = phi - (theta1_rad + theta2_rad)

        return (
            round(math.degrees(theta1_rad), 2),
            round(math.degrees(theta2_rad), 2),
            round(math.degrees(theta3_rad), 2)
        )

if __name__ == "__main__":
    arm = RoboticArm3DOF(L1=10.0, L2=8.0, L3=5.0)
    x, y, phi = arm.forward_kinematics(35, 45, -30)
    print(f"Forward Kinematics -> End-Effector: X={x} cm, Y={y} cm, Phi={phi}°")
    target = (12.79, 17.45, 50.0)
    sol = arm.inverse_kinematics(*target)
    print(f"Inverse Kinematics -> Joint Angles: {sol}")

```